DELIVERABLE 2

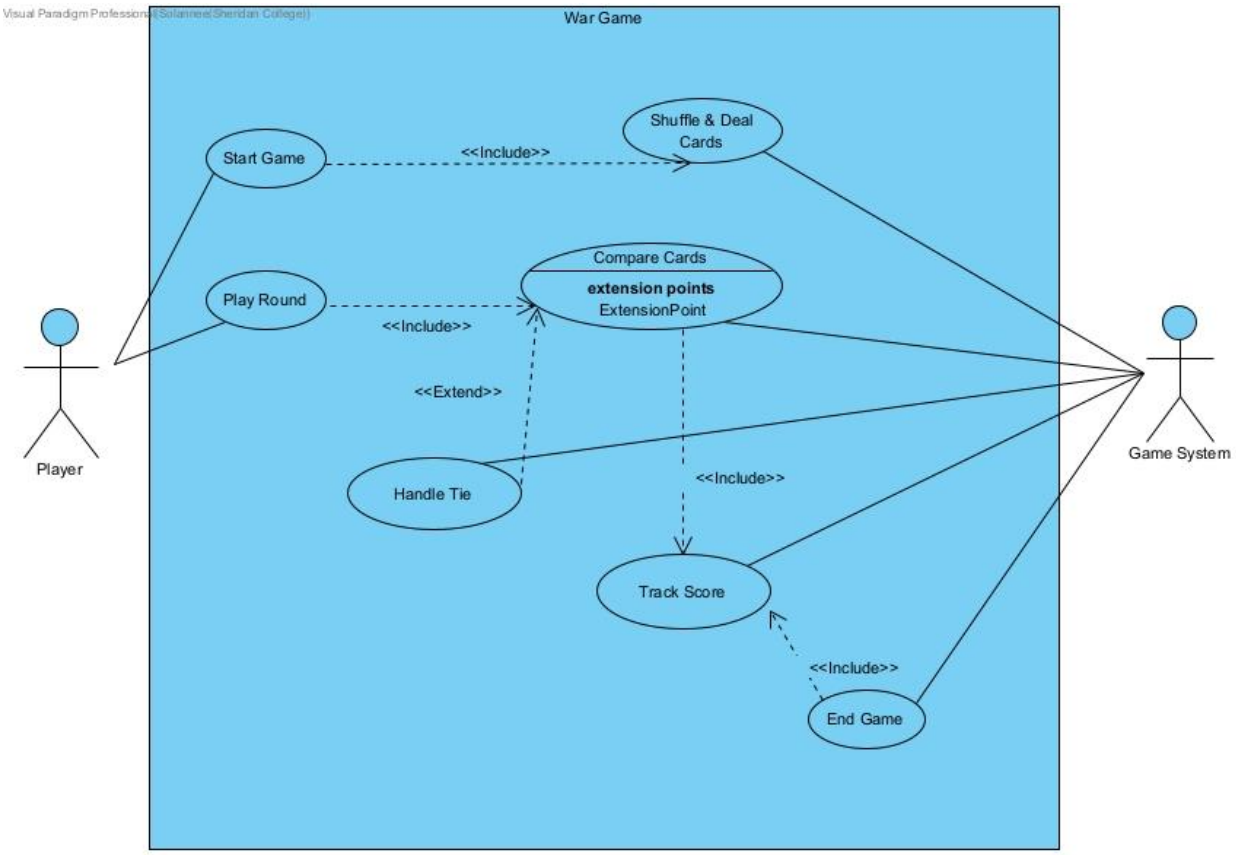
GAME MASTER (GROUP 7)

GROUP MEMBERS:

NEEL PARESH SOLANKI
VIMAL SHANTIBHAI LAKHANI
PRATHAM MILANKUMAR DOSHI
AZAAN SHAHZAD

TABLE OF CONTENT

USE CASE DIAGRAM (WAR GAME)



USE CASE NARRATIVES

1.1 Player starts the game.

1.2 Game system verifies player count and shuffles the deck.

1.3 Cards are evenly distributed among players.

1.4 Game system confirms game readiness.

1.5 Game begins.

2.1 Players play the top card from their deck.

2.2 Game system compares card values.

- 2.2.1 If one card is the highest, that player wins and collects all cards played.
- 2.2.2 If there is a tie, a War round is triggered.

3.1 Tied players place three face-down cards and play another card face-up.

3.2 Game system compares the new cards.

- 3.2.1 If one card is highest, the player wins and collects all cards.
- 3.2.2 If another tie occurs, the War process repeats.

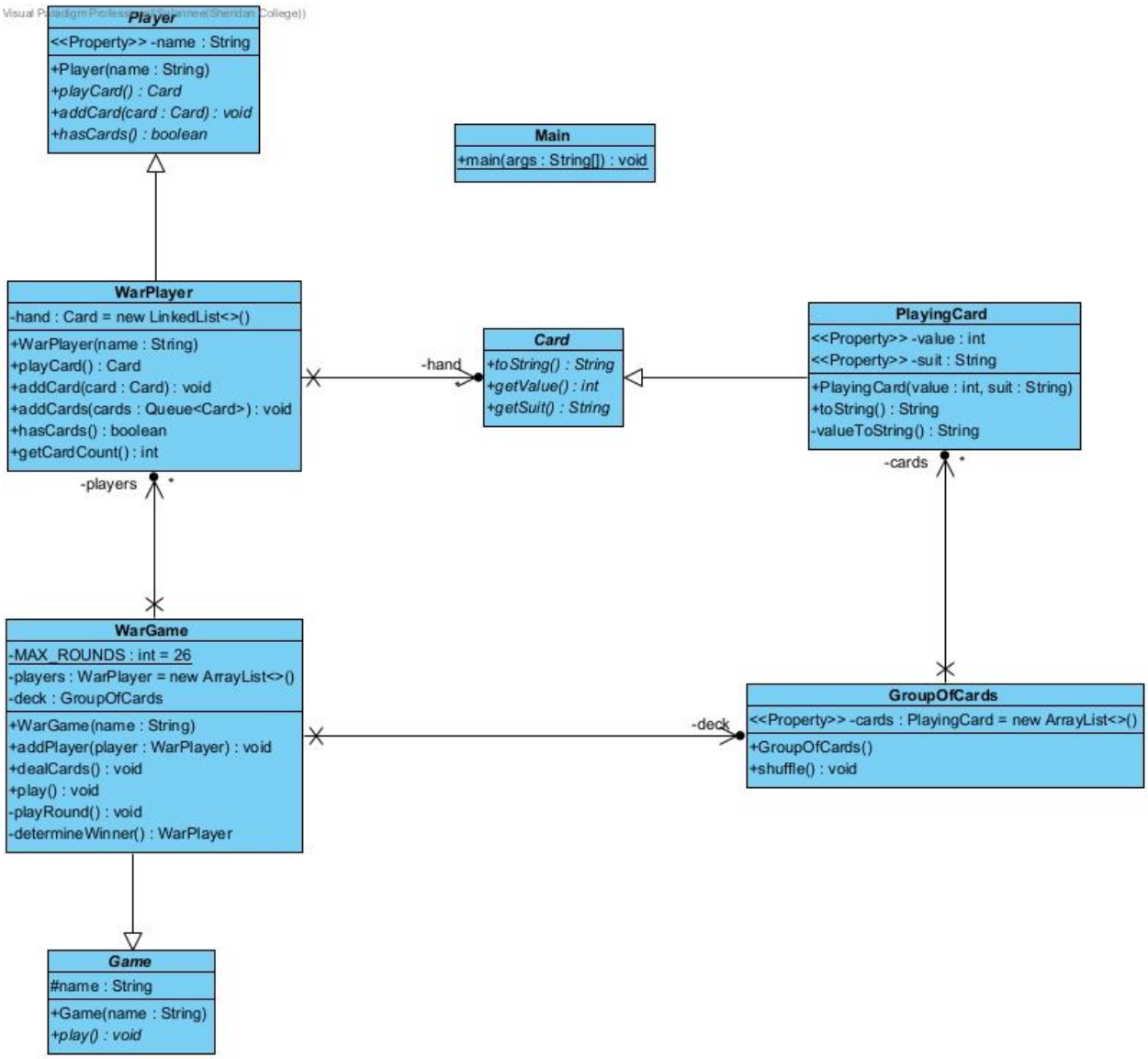
4.1 Game system updates player deck count and displays scores.

4.2 Game continues until 26 rounds are completed.

4.3 The player with the most cards wins.

4.4 Game system displays results and ends the session.

CLASS DIAGRAM



DESIGN DOCUMENT TEMPLATE

OVERVIEW

1. Project Background and Description

The chosen project implements a basic version of War Card Game. The game classifies as a turn-based two-player competition since each play round begins with players drawing the top card from their decks followed by a winner determination through card values. The game requires a War round between tied players to resume the comparing of additional placed cards to determine the winner. A multi-player system across 26 defined rounds constitutes the game scope. The winner of War Game becomes the final victor by accumulating the most cards during the 26 rounds. The program runs on Java through implementation which follows Object-Oriented Programming (OOP) design principles to maintain encapsulation, inheritance, and modular design rules.

2. Design Considerations

The main components of the system can be found in Class Diagram. WarGame extends Game through inheritance as the basis to manage game operations. The abstract Player class receives implementation through WarPlayer which controls specific actions and state of each player as well as their respective card decks. The Card class exists as an abstract entity which PlayingCard implements to serve as a standard playing card configuration with its value and suit components. The WarGame system establishes an aggregation with WarPlayer while WarPlayer controls their hand using a composition approach as a Queue<Card>. GroupOfCards functions as the game administrator to control the entire deck even though the game performs its initial card initialization functions through it.

- **Encapsulation** - The application defines all data fields as private members but provides public methods for authorized access. The PlayingCard object provides both getValue() and getSuit() methods but denies external users the ability to modify the data.
- **Delegation** - Responsibilities are well-distributed across classes. The WarGame system utilizes GroupOfCards to distribute cards and WarPlayer takes care of the activities related to playing and receiving cards by individual users.
- **Cohesion** - The classes have clearly defined functionalities which they execute independently. The WarGame class controls game management functions while WarPlayer maintains player card distributions and PlayingCard serves as a data storage unit for representative card details. The high cohesion brings both logical structure and simplified programming to the system.
- **Coupling** - Ongoing relationship strength remains minimal because base classes known as Player and Card abstract their functions from the system. Interface decoupling between classes helps make systems more testable and enhances their module independence.
- **Inheritance** - Inheritance is used appropriately. The game engine design adopts inheritance relations between Game and WarGame while establishing WarPlayer as a sub-class of Player so base operations can be utilized and expanded.
- **Aggregation** - The WarGame class maintains a list of independent WarPlayer objects allowing its existence without being dependent on the game object.
- **Composition** - The application achieves composition by using Queues to gather card storage for its players within WarPlayer. The object boundary of the hand stretches only across the extent of the persisting player entity
- **Flexibility/Maintainability** - The system is designed to be modular and easily extendable. New features such as score tracking enhancements, additional game modes, or user interface layers could be added in the future without altering the core logic. The use of abstraction and clearly separated responsibilities ensures that the codebase remains maintainable as the project evolves